Методические указания к лабораторной работе

Автор: доцент НИТПУ Друки А.А.

Работа с данными с помощью библиотеки Keras

Данная лабораторная работа посвящена получению навыков работы с табличными данными, их предобработки и классификации с помощью нейронных сетей, изучение функционала библиотеки машинного обучения Keras.

Цель работы: получить навыки решения задач многоклассовой классификации табличных данных с использованием искусственных нейронных сетей в библиотеке машинного обучения Keras.

Задачи:

1. Получение навыков работы с табличными данными с помощью библиотек Sklearn и Pandas;
2. Получение навыков разработки полносвязных моделей нейронных сетей с помощью библиотеки машинного обучения Keras.

Библиотека Keras

Keras – открытая нейросетевая библиотека, написанная на языке Python. Официальный сайт: https://keras.io/ Нацелена на оперативную работу с нейронными сетями, при этом обладает компактностью, модульностью и расширяемостью. Представляет собой надстройку над более низкоуровневыми фреймворками TensorFlow и Theano.

Библиотека Keras следует лучшим методам снижения когнитивной нагрузки: он предлагает последовательные и простые API-интерфейсы (программный интерфейс приложения, интерфейс прикладного программирования), сводит к минимуму количество действий пользователя, необходимых для типичных случаев использования, и предоставляет четкие и действенные сообщения об ошибках. Он также содержит обширную документацию и руководства для разработчиков.

Данная библиотека используется CERN, NASA, NIH и многими другими научными организациями для решения задач по машинному обучению во всем мире. Keras обладает низкоуровневой гибкостью для реализации произвольных исследовательских идей, предлагая дополнительные высокоуровневые удобные функции для ускорения циклов экспериментов. Благодаря простому использованию и ориентированию на пользователя, Keras является предпочтительным решением для глубокого обучения для многих университетских курсов. Он широко рекомендуется как один из лучших способов изучения глубокого обучения.

Ход работы

1. Загрузка данных

В качестве примера решим задачу многоклассовой классификации с помощью полносвязной нейронной сети.

В качестве набора данных возьмем базу оценки уровня физического развития людей разного возраста: https://www.kaggle.com/datasets/kukuroo3/body-performance-data

Данные представлены в табличном виде и содержат 12 параметров физического развития: возраст, пол, рост, вес, процент жира, количество приседаний, количество наклонов, длинна прыжка и т.д. Последний параметр – это уровень физического развития в виде класса (A,B,C,D).

Для начала работы импортируем необходимые библиотеки и модули.

Библиотека **matplotlib** предоставляет набор функций для создания статических, анимационных и интерактивных графиков на Python.

Библиотека **pandas** предоставляет инструменты для работы с данными в формате таблиц (DataFrame).

Библиотека **scikit-learn** предоставляет простой и эффективный инструментарий для анализа данных и построения моделей машинного обучения.

Импортирует функцию **train_test_split** из библиотеки **scikit-learn**, которая позволяет разделить набор данных на обучающую и тестовую выборки.

Импортирует методы **StandardScaler** и **LabelEncoder**. StandardScaler используется для нормализации данных, а LabelEncoder – для кодирования категориальных переменных в числовой формат.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
import matplotlib.pyplot as plt
import pandas as pd
```

Для загрузки датасета перетащите файл **bodyPerformance.csv** в левую панель Google Colab. Далее загрузим данные из файла и сохраним их в объекте DataFrame под названием **data**.

```
data = pd.read_csv('bodyPerformance.csv')
```

Выполним предварительную обработку данных. В переменную **X** сохраним данные из датасета **data**, без столбца **class**. Это будет набор признаков для обучения модели. В переменную **y** сохраним данные из столбца **class**. Это будет целевая переменная, которую модель будет предсказывать.

```
X = data.drop('class', axis=1)
y = data['class']
```

Выполним кодирование категориальных признаков. С помощью метода **LabelEncoder** преобразуем признак (M – male, F – female) в столбце **gender** в числовой формат (0, 1). Результаты сохраняются обратно в столбец **gender** в **X**.

```
label_encoder = LabelEncoder()
X['gender'] = label_encoder.fit_transform(X['gender'])
```

Тоже самое сделаем для переменной **y**. Создадим новый экземпляр **LabelEncoder** для переменной **y** и преобразуем её значения (A, B, C, D) в числовой формат (1, 2, 3, 4).

```
.
.
```

```
Допишите код
.
.
```

Для обучения и тестирования модели неросети необходимо общий дадасет разделить на обучающую и тестовую выборки. Для этого используем функцию **train_test_split** из библиотеки **scikit-learn**. На обучающую выборку выделим 80% от общего датасета, на тестовую 20%. Тоже самое сделаем для валидационной выборки.

```
.
.
Допишите код
.
.
```

С помощью метода **StandardScaler** выполним нормализацию данных вычисляя среднее значение и стандартное отклонение по каждому признаку.

```
.
.
Допишите код
.
.
```

2. Реализация топологии нейронной сети

Для создания модели нейронной сети будем применять библиотеку машинного обучения **Keras**.

Для этого нам потребуется импортировать и объявить класс **Sequential**, который позволяет создавать нейронные сети полносвязного типа, состоящие из последовательного стека слоев.

Импортируем метод **Dense**, который реализует полносвязный слой нейронной сети. Метод **Dense** содержит следующие параметры: количество нейронов, **input_shape** – формат входных данных (количество чисел, поступающих на вход нейросети), **activation** – функция активации.

Слои нейросети добавляются последовательно друг за другом путем вызова метода **add()**.

```
import keras
from keras.models import Sequential
from keras.layers import Dense
.
.
Допишите код
.
.
```

Таким образом у нас получилась нейронная сеть, состоящая из нескольких последовательных слоев. Связи между нейронами в слоях выстраиваются автоматически по принципу «каждый с каждым».

3. Подготовка модели к процессу обучения

После того, как модель была объявлена и собрана необходимо её скомпилировать с помощью метода **compile**.

```
.
.
Допишите код
.
.
```

Compile – это метод для компиляции нейронной сети, при котором выбираются: функция ошибки (loss), метрики оценки качества по умолчанию (accuracy), алгоритм обучения (optimizer).

Loss – функция ошибки нейронной сети – математическая дифференцируемая функция, характеризующая разницу между «истинным» значением целевой переменной и предсказанным нейронной сетью значением.

Sparse categorical crossentropy – (разреженная категориальная кросс-энтропия) представляет собой метод для измерения разницы между предсказанными вероятностями и истинными метками классов в задачах многоклассовой классификации. В отличие от обычной категориальной кросс-энтропии (categorical crossentropy), которая требует, чтобы метки классов были представлены в виде вектора one-hot (например: класс 0 – [1, 0, 0], класс 1 – [0, 1, 0], класс 2 – [0, 0, 1]). Sparse categorical crossentropy позволяет использовать целочисленные метки (например: класс 0 – 0, класс 1 – 1, класс 2 – 2). Это упрощает задачу подготовки данных.

Metrics – список метрик для вывода (точность 'accuracy' или потери 'loss'), используемых при обучении и тестировании.

Optimizer – (оптимизатор) алгоритм обучения, задача которого – изменение весов сети для уменьшения ошибки сети в процессе обучения.

Adam – оптимизационный алгоритм, используемый для обучения нейронной сети, основная функция оптимизатора – изменение весов для уменьшения ошибки сети в процессе обучения. Для каждого нейрона алгоритм изменяет веса индивидуально.

4. Обучение нейронной сети

Для обучения нейронной сети необходимо вызвать метод .**fit()**, используя блок кода ниже:

```
.
.
Допишите код
.
.
```

fit() – метод Keras, запускающий обучение нейронной сети.

Используются следующие параметры:

x_train – входные данные (изображения из обучающей выборки).

y_train – целевые данные (классы входных изображений).

epochs – количество повторений циклов обучения для всей выборки данных.

5. Тестирование нейронной сети

Для тестирования нейронной сети необходимо запустить следующий блок кода:

```
.
.
Допишите код
.
.
```

evaluate() – метод оценки модели на тестовых данных.
X_test – входные данные (изображения из тестовой выборки).
y_test – целевые данные (классы входных изображений).

6. Результат работы нейронной сети

Для визуализации процесса обучения построим графики и выведем их на экран.

```
print(f'Architecture {model}: Test Loss: {loss}, Test Accuracy: {accuracy}')
plt.figure(figsize=(18, 6))

plt.subplot(1, 3, 1)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title(f'Architecture {model} Loss')
plt.legend()

plt.subplot(1, 3, 2)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title(f'Architecture {model} Accuracy')
plt.legend()

plt.tight_layout()
plt.show()
```

На рисунке 1 представлена визуализация процесса обучения в виде графиков. Как можно видеть по графикам в процессе обучения нейронной сети ошибка постепенно уменьшается, а точность увеличивается.

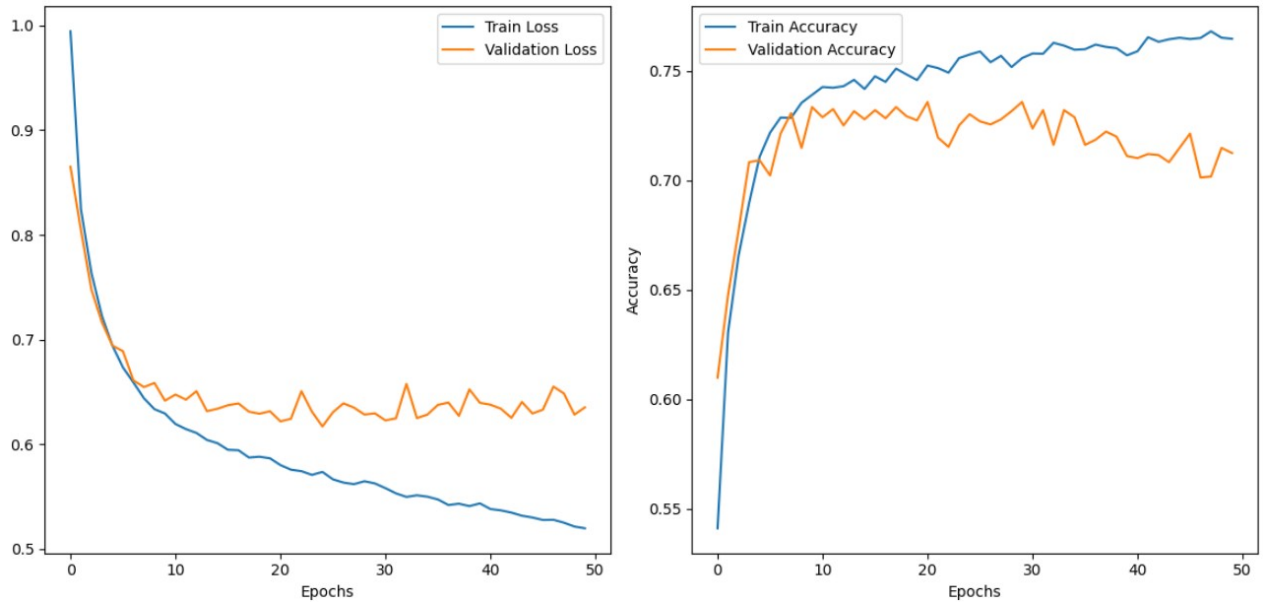


Рисунок 1. Визуализация процесса обучения нейронной сети

Задание

В качестве заданий для самостоятельной работы выберите один вариант из представленных ниже:

1. Оценка вероятности диагностики диабета у человека: https://www.kaggle.com/datasets/alexteboul/diabetes-health-indicators-dataset Метка класса: Diabetes_012. Класс отрицательный – 0 (no diabetes – нет диабета), класс положительный – 1; 2 (prediabetes – преддиабетическое состояние; diabetes - диабет).

2. Оценка вероятности, того, что клиент откроет банковский депозит в результате маркетинговой акции: https://archive.ics.uci.edu/ml/datasets/Bank+Marketing Метка класса: y. Класс отрицательный – 'yes', Класс положительный – 'no'.

3. Оценка уровня физического развития людей разного возраста: https://www.kaggle.com/datasets/kukuroo3/body-performance-data Метка класса: class. Класс отрицательный – 1 (Классы A и B), класс положительный – (Классы C и D).

4. Оценка вероятности пожара по детекторам физических характеристик среды: https://www.kaggle.com/datasets/deepcontractor/smoke-detection-dataset Метка класса: Fire_Alarm. Класс отрицательный – 0 (нет огня), класс положительный – 1 (пожар).

Вывод

В результате выполнения данной лабораторной работы Вы научились создавать полносвязные нейронные сети с помощью библиотеки Keras для решения задачи

классификации табличных данных. Так же в рамках данной работы были получены навыки работы с Google Colab, iPython Notebooks.

Критерии оценки

Максимальное число баллов, которое можно получить по лабораторной работе – 8 баллов.

Минимальное число баллов, при котором задание засчитывается – 5 баллов.

Выполнение задания

Выполнено меньше половины задания - 0 баллов.

Большей частью выполнено - 2 балл.

Выполнено полностью - 4 балла.

Качество отчета

Сильные огрехи в оформлении или удовлетворяет менее половины требований к отчету - 0 баллов.

Хорошее оформление. Удовлетворяет большей части требований - 2 балл.

Соответствует всем требованиям - 4 балла.

Примечание

Предобработка датасета для варианта № 2. Так значения многих столбцов представлены в виде текста, то необходимо их все перевести в цифровой категориальный вид.

```
# Загрузка данных
data = pd.read_csv('bank-full.csv', sep=';')

# Выбор категориальных столбцов для кодирования
categorical_columns = ['job', 'marital', 'education', 'default', 'housing',
'loan', 'contact', 'month', 'poutcome']

# Применение OneHotEncoder к категориальным столбцам
encoder = OneHotEncoder(sparse_output=False)
encoded_columns = encoder.fit_transform(data[categorical_columns])

# Создание нового DataFrame с закодированными признаками
encoded_df = pd.DataFrame(encoded_columns,
columns=encoder.get_feature_names_out(categorical_columns))

# Объединение закодированных признаков с остальными данными
data = pd.concat([data.drop(columns=categorical_columns), encoded_df],
axis=1)
```

```
# Предварительная обработка данных
X = data.drop('y', axis=1)
y = data['y']

# Кодирование целевой переменной
label_encoder_y = LabelEncoder()
y = label_encoder_y.fit_transform(y)

# Разделение на обучающую, тестовую и валидационную выборки
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train,
test_size=0.2)

# Нормализация данных
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test)
```